Cross Asset Research Suite

Whitepaper: design rationale, best practices, and operating framework for cross-asset analysis in TradingView

Prepared for the custom Pine v6 indicator implementation

Version note: based on the fixed Pine v6 script and current TradingView documentation available in April 2026.

At a glance	Summary
Purpose	Provide a practical research framework for comparing two assets inside TradingView using a single Pine v6 indicator.
Core modes	Mean reversion, relative strength / leadership, and spread monitoring.
Best fit	Liquid, economically linked pairs such as stock vs benchmark ETF, sector ETF vs broad market, or commodity vs proxy.
Not designed for	Formal cointegration testing, portfolio construction, or automated execution.

Cross Asset Research Suite (XARS) is a two-asset analysis indicator for TradingView that combines rolling correlation, normalized performance, ratio and spread models, z-score stretch, and a simple leadership proxy into a single decision framework. The intent is not to replace statistical arbitrage infrastructure; it is to make disciplined cross-asset comparison usable inside a charting workflow.

> **Positioning:** The program is best understood as a research and decision-support layer. It helps you screen relationships, visualize them consistently, and create alertable rules, but it does not prove a durable equilibrium relationship by itself.

Executive summary

The indicator is built around several current Pine best practices. Pine v6 enables dynamic request.*() behavior by default, which allows symbol inputs to feed request.security() directly. At the same time, TradingView's documentation still recommends using request.security() mainly for the chart timeframe or higher timeframes, and using request.security_lower_tf() when true lower-timeframe intrabar arrays are required.[1][2]

XARS therefore keeps the core pair engine on the chart timeframe, uses grouped inputs for fast reconfiguration, updates its dashboard on the last bar, and places selected markers on the main chart while the analytics remain in a dedicated pane.[4][5][9]

Conceptually, the indicator blends two families of cross-asset thinking. One is the classical distance/relative-value view, where normalized prices or spreads are compared over a lookback. The other is the modern charting view, where correlation, leadership, and alertable thresholds guide discretionary or semi-systematic decisions. The result is a tool that can be used for pair-trading research, benchmark-relative leadership, sector rotation work, or proxy analysis.

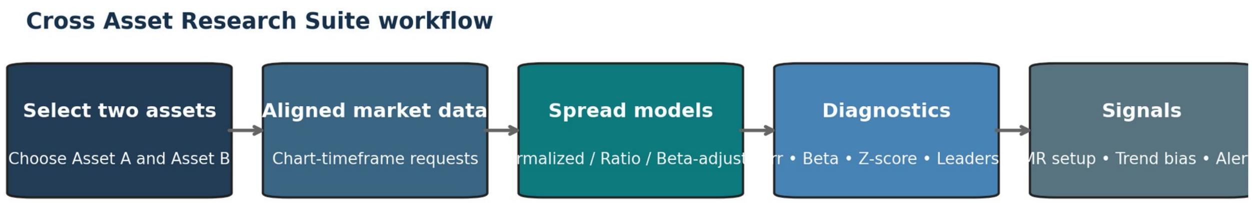

Figure 1. Core workflow of the Cross Asset Research Suite.

1. Research basis and design principles

1.1 Same-timeframe first, higher-timeframe only when deliberate

TradingView's own guidance is clear: request.security() is intended for equal or higher timeframes, while lower-timeframe use through request.security() returns only one lower-timeframe bar per chart bar. If the user truly needs all intrabars, request.security_lower_tf() is the correct interface. For a two-asset relationship tool, the safest default is therefore to analyze the pair on the chart timeframe and simply change the chart to the timeframe you want to study.[2]

1.2 Correlation is a filter, not proof

The indicator uses fast and slow rolling correlations on log returns because correlation remains useful as a screening and regime filter. TradingView's CC documentation describes the familiar scale from +1 to -1, where positive values indicate co-movement and negative values indicate opposing movement. In practice, a strong positive slow correlation helps justify mean-reversion or relative-value monitoring; a collapsing correlation is a warning that the pair relationship is changing.[7]

However, correlation alone does not prove that a spread is stable or tradable. This is why the script treats correlation as a gate and not as a sufficient condition. The whitepaper's operating guidance assumes that users still apply economic judgment when choosing the pair.

1.3 Distance logic still matters

The normalized-performance option is rooted in the older relative-value literature. Gatev, Goetzmann, and Rouwenhorst matched stocks by minimum distance in normalized historical price space. XARS does not replicate their full portfolio construction process, but it intentionally preserves the intuition: if two related assets have historically tracked each other and the normalized distance widens, the divergence may deserve attention.[10]

1.4 Efficient Pine architecture

The implementation also follows Pine efficiency guidance. TradingView caps scripts at 40 unique request.*() calls on most plans and 64 on Ultimate, and the profiling guidance recommends minimizing these requests and combining same-context expressions into tuple-based calls. XARS requests several series from one context in single tuple calls, uses calc_bars_count to avoid waste, and keeps the total request footprint small.[6][8]

2. Indicator architecture

Layer	What the script does	Why it matters
Inputs	Collects Asset A, Asset B, source mode, spread model, thresholds, and display choices.	Makes the tool reconfigurable without editing code.
Data requests	Requests the selected source from both assets on the chart timeframe using request.security().	Keeps the pair aligned to one consistent bar set.
Core analytics	Computes returns, correlation, rolling beta, normalized series, ratio/spread models, z-score, momentum, and lead/lag proxy.	Creates several ways to understand the relationship.
Signals	Defines mean-reversion, trend-leadership, lead/lag, and correlation-breakdown conditions.	Lets the user separate reversal setups from trend continuation.
Visuals	Plots one view at a time in a separate pane, shades regimes, and optionally draws markers on the main chart with force_overlay.	Reduces clutter while keeping trade cues visible on price.
Dashboard + alerts	Summarizes state in a table and exposes alertcondition() triggers.	Supports a repeatable workflow and TradingView-native alerts.

The mixed-pane design is intentional. TradingView allows a script to live in a separate pane while selected elements are overlaid on the main chart through force_overlay. This makes it possible to keep statistics readable without hiding price action.[4]

3. Analytical engine

3.1 Returns and rolling correlation

The script starts with log returns for each asset. Fast and slow rolling correlations are then computed on those return streams. Fast correlation is a short-term temperature check; slow correlation is the regime filter used by the mean-reversion engine.

```
retA = ln(A_t / A_{t-1})
retB = ln(B_t / B_{t-1})
corrFast = corr(retA, retB, corrFastLen)
corrSlow = corr(retA, retB, corrSlowLen)
```

3.2 Rolling beta

A rolling beta is estimated from return covariance and variance. In the fixed version of the script, covariance is computed manually from rolling means because ta.covariance() is not exposed as a built-in series function in the environment where the script is compiled. This beta is then used in the beta-adjusted spread model.

```
meanRetA = sma(retA, betaLen)
meanRetB = sma(retB, betaLen)
covAB    = sma(retA * retB, betaLen) - meanRetA * meanRetB
varB     = sma(retB * retB, betaLen) - meanRetB * meanRetB
beta     = covAB / varB
```

3.3 Four spread models

Model	Formula / intuition	Best use case
Normalized performance difference	Index each asset to 100 over a lookback, then subtract.	Classic relative-value visual; useful when the user wants intuitive distance behavior.
Price ratio	A_t / B_t.	Simple leadership and ratio-trend work when absolute scale differences are acceptable.
Log ratio	ln(A_t) - ln(B_t).	Cleaner than a raw ratio when price levels differ materially.
Beta-adjusted log spread	ln(A_t) - beta × ln(B_t).	Best default when the user wants a more hedge-aware spread.

The reason to offer multiple models is practical. Some users are looking for a classical pair spread that may revert. Others are simply asking which asset is outperforming. One instrument can support both styles if the user can switch the spread engine cleanly.

3.4 Z-score as a decision axis

Whatever the chosen core series is, the indicator standardizes it into a z-score over a rolling window. This makes the alert thresholds portable: a value above +2.0 means the spread is far above its recent mean in standardized terms, while a value below -2.0 means it is far below.

```
coreMean = sma(coreSeries, zLen)
coreStd  = stdev(coreSeries, zLen)
zScore   = (coreSeries - coreMean) / coreStd
```

3.5 Leadership view

Leadership in XARS is not defined by one number. It is a composite reading from the fast/slow EMA structure of the price ratio, the momentum of that ratio, and a lead/lag proxy built from offset return correlations. The lead/lag score should be treated as a heuristic rather than a causal claim.

```
ratioFast    = ema(priceRatio, ratioFastLen)
ratioSlow    = ema(priceRatio, ratioSlowLen)
emaSpreadPct = 100 * (ratioFast / ratioSlow - 1)
rsMomentum   = roc(priceRatio, rsMomLen)
leadLagScore = corr(retA[1], retB, L) - corr(retA, retB[1], L)
```

4. Signal logic

The indicator separates reversal logic from trend logic. This is important because many users mix them accidentally: a pair can have a stretched spread that looks tempting to fade, while the ratio trend and momentum still argue that one asset is leading decisively.

Signal family	Trigger	Interpretation
MR Long A / Short B	z-score crosses below the negative entry threshold while slow correlation remains above the minimum.	Asset A has underperformed unusually far relative to Asset B within a still-acceptable relationship.
MR Long B / Short A	z-score crosses above the positive entry threshold while slow correlation remains above the minimum.	Asset B has underperformed unusually far relative to Asset A.
MR Exit	Absolute z-score returns inside the exit band.	The spread has substantially normalized.
A leadership cross	Fast ratio EMA crosses above slow ratio EMA with positive ratio momentum.	A is taking relative-strength leadership.
B leadership cross	Fast ratio EMA crosses below slow ratio EMA with negative ratio momentum.	B is taking relative-strength leadership.
Correlation breakdown	Slow correlation drops below the minimum threshold.	Treat pair-trading logic with caution; the relationship may be changing.

> **Operating rule:** A good discipline is to choose one regime first. If you are trading the indicator as a mean-reversion tool, let the correlation and z-score drive the decision. If you are trading it as a leadership tool, let the ratio structure and momentum drive the decision.

5. Best practices for real-world use

5.1 Choose economically linked pairs

- Stock versus benchmark or sector ETF: AAPL vs QQQ, XLK vs SPY, XLF vs SPY.
- Sector proxy pairs: XLE vs SPY, SMH vs QQQ.
- Commodity or macro proxy pairs: use only when the economic link is clear and the trading session overlap is understood.

- Avoid thin or irregular instruments. Relationship tools degrade quickly when either leg is illiquid or structurally noisy.

5.2 Match the chart timeframe to the decision horizon

Because the script deliberately keeps the analysis on the chart timeframe, the chart is the control surface. If you want an intraday pair read, put the chart on 5-minute or 15-minute bars. If you want a swing read, use daily or weekly bars. This design choice is aligned with TradingView's guidance on request.security() and avoids lower-timeframe ambiguity.[2]

5.3 Do not confuse a screen with a proof

- High correlation does not guarantee mean reversion.
- A stable spread in one lookback does not guarantee stability after earnings, macro releases, or commodity roll changes.
- The lead/lag proxy is exploratory. Use it to organize attention, not to infer causality.
- Backtesting and position-sizing logic still belong in a strategy or external research stack.

5.4 Confirm signals on bar close when reliability matters

The indicator includes a bar-close confirmation option. This is usually the cleaner choice when alerts or study notes need to match what the chart shows after the bar is complete. Alertcondition() merely exposes triggers; the user must still create the actual TradingView alerts from the Create Alert dialog. [3]

5.5 Keep request usage efficient if you extend the script

If the script is later expanded to more symbols or more requested series, follow TradingView's efficiency rules: keep request.*() counts low, condense same-context expressions into tuples, and remember the global tuple-element limit. These details matter quickly in cross-asset screeners.[6][8]

6. Suggested operating presets

Use case	Spread model	Chart timeframe	Useful starting settings
Stock vs benchmark ETF	Beta-adjusted log spread	5m, 15m, or 1D	corrSlow 60, zLen 60, meanRevEntryZ 2.0, meanRevExitZ 0.5
Sector ETF vs SPY	Normalized performance difference	15m or 1D	normLookback 60, corrSlow 60, ratio EMAs 10/30
Relative-strength leadership	Price ratio or log ratio	5m to 1D	focus on ratioFast/ratioSlow, rsMomLen 10, leadLagLen 20
Research / screening	Switch between normalized and beta-adjusted spread	1D	use slow correlation as the primary gate before any signal interpretation

These are deliberately conservative presets. They are meant to stabilize the reading before the user starts optimizing every slider. Most bad outcomes come from forcing a noisy pair into a mean-reversion template, not from slightly imperfect parameter values.

7. Limitations and boundaries

- The program is an indicator, not a strategy. It exposes alert triggers but does not backtest or place hypothetical orders.
- The program does not perform formal cointegration tests, residual diagnostics, or hedge-ratio rebalancing beyond a rolling beta approximation.
- Cross-asset comparisons can be distorted around earnings, splits, contract rolls, and session mismatches.
- Commodity-vs-equity or futures-vs-ETF comparisons often require stronger discretionary filtering than equity-vs-equity relationships.
- Any extension that requests many more symbols or many more datasets must respect TradingView request.*() limits and performance guidance.

8. Conclusion

Cross Asset Research Suite is most powerful when treated as a disciplined charting framework rather than a black box. Its strengths are speed, clarity, and a sensible Pine architecture: same-timeframe analysis, flexible spread models, standardized thresholds, leadership diagnostics, chart-visible cues, and TradingView-native alerts. Used this way, it becomes a repeatable bridge between visual chart analysis and structured relative-value research.

References

Accessed April 2026.

[1] TradingView User Manual — Other timeframes and data — Dynamic requests in Pine v6, request.security(), and related request.*() behavior.

[2] TradingView FAQ — Other data and timeframes — Lower-timeframe limitations, request.security_lower_tf(), and the recommended higher-timeframe non-repainting pattern.

[3] TradingView FAQ — Alerts — alertcondition() behavior and the need for users to create alerts from the Create Alert dialog.

[4] TradingView User Manual — Visuals overview — overlay and force_overlay behavior for mixed-pane designs.

[5] TradingView User Manual — Tables — Best practice for declaring tables with var and updating them on the last bar.

[6] TradingView User Manual — Limitations — request.*() call limits, tuple element limits, and calc_bars_count behavior.

[7] TradingView Help Center — Correlation Coefficient (CC) — Meaning of the correlation scale from -1 to +1 in chart analysis.

[8] TradingView User Manual — Profiling and optimization — Guidance on minimizing request.*() calls and using tuple requests for efficiency.

[9] TradingView User Manual — Inputs — input.symbol(), input.* grouping, and user-facing configuration design.

[10] Gatev, Goetzmann, and Rouwenhorst (2006), Pairs Trading: Performance of a Relative-Value Arbitrage Rule — Classic reference for distance-based pair selection using normalized historical prices.